

FIBRAS-mx

Experimento 2: Algoritmo

Dos sub-experimentos independientes con el mismo split de datos

Autor:	Daniel Becerril Olguín
Fecha:	Mayo 2026
Stack:	Python 3.12 · scikit-learn · XGBoost · CatBoost · LightGBM · Streamlit
Datos:	2017-2026 · 15 FIBRAS · frecuencia diaria

Sub-experimento 2A — ML Puro

Entrenar 9 modelos de Machine Learning (Decision Tree, Ridge, Logistic Regression, Random Forest, XGBoost, CatBoost, LightGBM, Extra Trees, ElasticNet) sobre el universo completo de hasta 400 features. El modelo aprende qué señales predicen mejor el retorno de cada FIBRA. No hay GA ni selección manual de features.

Sub-experimento 2B — Algoritmo Genético

Buscar en el espacio de 40 variables $\times$ 10 parametrizaciones = 400 genes posibles, eligiendo combinaciones de 1 a 5 genes. El cromosoma ES la estrategia: define una regla de scoring directa que selecciona las top-k FIBRAS cada trimestre. Ningún modelo ML interviene en la evaluación.

0. Datos compartidos por ambos sub-experimentos

Frecuencia: DIARIA — no solo trimestral

Ambos sub-experimentos comparten el mismo dataset. Se usa frecuencia DIARIA: ~26,250 observaciones en train (vs ~420 si usáramos solo rebalanceos trimestrales).

- Precios OHLCV: diarios → features técnicas calculadas día a día.
- Fundamentales trimestrales: forward-fill diario entre reportes.
- Rebalanceo: sigue siendo trimestral (ejecución igual que E0-E13).

Partición de datos

Split	Período	Días hábiles	Obs. (días×15)	Propósito
Train	2017-01 → 2023-12	~1,750	~26,250	GA + ML
Test	2024-01 → 2025-12	~500	~7,500	Evaluación
Validation	2026-01 → hoy	~90	~1,350	Holdout real

Universo de features: 40 variables × 10 parametrizaciones = 400 features

Bloque	Origen	Vars	Ejemplo de parametrización
A (15 vars)	Derivadas de E0-E13 (retorno, vol, MA, yield)	×10	ret trailing: 21d, 42d, 63d, 84d, 126d, 168d, 252d, 378d, 504d, 756d
B (5 vars)	Fundamentales nuevas (NOI, deuda/activos, dilución)	×10	NOI margin: raw, rank, >30%, >40%, >50%, >60%, >65%, QoQ, YoY, trend4Q
C (5 vars)	Técnicas nuevas (RSI, Bollinger, z-vol, 52w-h)	×10	RSI: 7, 9, 14, 21, 28, 42, 63, <30(oversold), >70(overbought), dist-50
D (5 vars)	Tendencia nuevas (aceleración, multi-TF, consistencia)	×10	Score multi-TF: 8 combos de ponderaciones distintas entre 1M/3M/6M/12M
E (5 vars)	Mercado nuevas (fuerza relativa, sector, corr)	×10	Fuerza vs índice: ret-ret_index (21d, 63d, 126d, 252d), rank, >0%, >5%...
F (5 vars)	Macro nuevas (tasa real, spread CETES, USD/MXN, IPC)	×10	Spread yield-CETES: >0%, >1%, >2%, >3%, >4%, zscore, rank, tendencia...

Sub-experimento 2A — ML Puro

Se entrena cada uno de los 9 modelos sobre el conjunto completo de features (hasta 400 columnas). No hay selección previa de variables — el modelo aprende qué pesa. La tarea es ranking: predecir qué FIBRAS tendrán mejor retorno relativo en los próximos 63 días (1 trimestre).

Pipeline

1. Construir feature matrix diaria completa (hasta 400 columnas)
 - Features técnicas: día a día desde precios OHLCV
 - Features fundamentales: forward-fill de datos trimestrales
2. Label: retorno de cada FIBRA a 63 días hacia adelante
 - Normalizado 0–1 con rank entre las 15 FIBRAS
 - Horizonte fijo: 63 días (1 trimestre = 1 período de rebalanceo)
3. Walk-forward CV dentro del train (ventana mín 252 días, paso 63 días):
Para cada fold t:
 - Entrenar modelo en días $\leq t$
 - Predecir scores diarios para $[t+1 \dots t+63]$
 - Agregar scores por FIBRA (promedio diario)
 - Seleccionar top-5 FIBRAS
 - Calcular retorno portfolio (igual peso, 63 días)
4. Evaluar en Test y Validation con modelo entrenado en todo el Train

9 Modelos con hiperparámetros

#	Modelo	Hiperparámetros clave
1	Decision Tree	max_depth [2,3,4,5,6], min_samples_split [2,5,10,20]
2	Ridge Regression	alpha [0.01, 0.1, 1, 10, 100]
3	Logistic Regression	C [0.01, 0.1, 1, 10], solver [lbfgs, liblinear]
4	Random Forest	n_estimators [50,100,200], max_depth [2,3,4,5,None]
5	XGBoost	n_estimators [50,100,200], max_depth [2,3,4], lr [0.01,0.1,0.3]
6	CatBoost	iterations [50,100,200], depth [2,3,4], lr [0.01,0.05,0.1]
7	LightGBM	n_estimators [50,100,200], max_depth [2,3,4], lr [0.01,0.1]
8	Extra Trees	n_estimators [50,100,200], max_depth [2,3,4,None]
9	ElasticNet	alpha [0.01,0.1,1], l1_ratio [0.1,0.5,0.9]

Output de la Sección 2A en Streamlit

- Tabla comparativa: Sharpe_train, Sharpe_test, CAGR_train, CAGR_test, MaxDD_test
- Equity curve del mejor modelo vs E0 (Naive 1/N), E13 y CETES
- Feature importance de los modelos que la soportan (RF, XGB, LGB)

Sub-experimento 2B — Algoritmo Genético

Búsqueda combinatoria sobre el espacio de 400 genes posibles. Un individuo elige entre 1 y 5 genes; cada gen es una (variable, parametrización). El cromosoma define directamente la regla de scoring: para cada FIBRA se suma el valor de sus genes seleccionados, se rankea, y se elige el top-k. No interviene ningún modelo de ML.

Encoding del cromosoma

```
Gene = namedtuple('Gene', ['var_id', 'param_idx'])
# var_id   : 0-39 (40 variables)
# param_idx: 0-9  (10 parametrizaciones)
# Espacio total: 400 genes posibles

@dataclass
class Individual:
    genes: list[Gene] # 1-5 genes, sin repetir var_id
    top_k: int        # FIBRAS a seleccionar: in {3,4,5,6,7}
    fitness: float = -inf
```

Función de Fitness (sin ML)

```
fitness(individual, train_data):
    Para cada fecha de rebalanceo t en el train:
        a. Para cada FIBRA: score = suma de gene_value[var_id][param_idx]
        b. Normalizar scores (rank 0-1 entre FIBRAS)
        c. Seleccionar top_k FIBRAS con mayor score
        d. Retorno del portfolio: igual peso, hold 1 trimestre

    Sharpe de la serie de retornos trimestrales
    Parsimony penalty:  $-0.02 \times (n\_genes - 1)$  ← menos genes = menos overfitting
    Retornar Sharpe ajustado
```

Parámetros del GA

- Población: 20 individuos
- Generaciones: 50 (configurable en UI, slider 10-100)
- Selección: torneo de tamaño 3, elitismo top-2
- Early stopping: sin mejora en 10 generaciones
- Re-diversificación: si >80% población idéntica, reiniciar 40% (immigration)

Operadores evolutivos

```
Inicialización:
n_genes = random.choice([1,2,3,4,5])
genes   = random.sample(ALL_400_GENES, n_genes) # sin repetir var_id
top_k    = random.choice([3,4,5,6,7])

Cruzamiento:
```

```
gene_pool = union(parent1.genes, parent2.genes)
n_child   = random.choice([1..5])
child_genes = random.sample(gene_pool, min(n_child, len(pool)))
child_top_k = random.choice([p1.top_k, p2.top_k])
```

Mutación (P=0.3 por tipo, independientes):

```
mutate_add:   agregar gene aleatorio (si n < 5)
mutate_remove: eliminar gene aleatorio (si n > 1)
mutate_swap:  reemplazar gene por otro (distinto var_id)
mutate_param: cambiar param_idx de un gene existente
mutate_topk:  cambiar top_k (P=0.2)
```

Output de la Sección 2B en Streamlit

- Gráfica de convergencia: mejor fitness y media poblacional por generación
- Cromosoma ganador decodificado: variables, parametrización, top_k
- Equity curve en Train / Test / Validation vs E0 y E13
- Frecuencia de cada variable en el top-10 de individuos finales

1. Detalle de features — Bloque A (estrategias existentes E0-E13)

ID	Variable	10 Parametrizaciones
A01	Retorno trailing	21d, 42d, 63d, 84d, 126d, 168d, 252d, 378d, 504d, 756d
A02	Log-retorno trailing	Mismos 10 horizontes que A01
A03	Volatilidad realizada	21d, 42d, 63d, 84d, 126d, 168d, 252d, 378d, 504d, ratio(21/252)
A04	Precio / MAN	MA20, MA30, MA50, MA60, MA100, MA120, MA150, MA200, MA250, MA300
A05	MAN > MAM (Golden Cross)	(20>50), (20>100), (50>100), (50>200), (20>200), (100>200), (30>100), (30>200), (60>150), (10>50)
A06	Fuerza relativa vs MA	Distancia % a MA50, MA100, MA200, sus negativos/invertidos + log
A07	Dividend yield TTM	raw, rank, >3%, >4%, >5%, >6%, >7%, >8%, >9%, top25%
A08	Precio / NAV	raw, rank, <0.80, <0.90, <0.95, <1.0, <1.05, <1.10, descuento%, rank_desc
A09	Ocupación	raw, rank, ≥0.75, ≥0.80, ≥0.85, ≥0.87, ≥0.90, ≥0.92, ≥0.95, QoQ-change
A10	FFO yield	raw, rank, >4%, >5%, >6%, >7%, >8%, >9%, >10%, anualizado×4
A11	Payout ratio (dist/FFO)	raw, rank, <0.60, <0.70, <0.75, <0.80, <0.90, <1.0, QoQ-change, inverted
A12	LTV ratio	raw, rank, <0.30, <0.35, <0.40, <0.45, <0.50, QoQ-change, inverted, >0.50
A13	Cambio Banxico 6M	raw, >+50bps, >+100bps, <-50bps, <-100bps, abs >50bps, dirección, aceleración, rolling12M, zscore
A14	Precio 30d mean	raw, rank, rank_top3, rank_top5, rank_top7, top50%, percentil, z-score, log, norm
A15	Sector dummy	industrial=1, hoteles=1, comercial=1, industrial+comercial, diversificado=1, no_hoteles=1, ponderado, mixto=1, 4 dummies

Bloques B-F (variables nuevas)

ID	Variable	10 Parametrizaciones
B01	NOI margin	raw, rank, >30%, >40%, >50%, >60%, >65%, QoQ-change, YoY-change, trend4Q
B02	Deuda / Activos	raw, rank, <0.35, <0.40, <0.45, <0.50, <0.55, QoQ-change, inverted, trend4Q
B03	Dilución de CBFIs	%change YoY, positivo, negativo, abs, <1%, <3%, <5%, <-1%, <-3%, z-score
B04	Apreciación propiedades	%change YoY, rank, >0%, >2%, >5%, >8%, <0%, trend4Q, QoQ-change, zscore
B05	Distribución / Activos	dist×cbfis/total_assets, rank, >1%, >2%, >3%, >4%, >5%, trend4Q, QoQ, z-score
C01	RSI	RSI-7, RSI-9, RSI-14, RSI-21, RSI-28, RSI-42, RSI-63, RSI<30, RSI>70, dist-de-50
C02	Posición Bollinger	BB(20,2), BB(20,1.5), BB(50,2), %B(0-1), %B<0.2, %B>0.8, BB-width, BB-width-z, squeeze, inv-%B
C03	Z-score volumen	z(21d), z(63d), z(126d), z(252d), >+1σ, >+2σ, <-1σ, tendencia, ratio(21/63), ratio(5/21)
C04	Precio / 52-week high	raw, >0.90, >0.95, >0.98, <0.80, <0.70, dist%, drawdown-52w, nuevo-máx, recuperación
C05	Amihud Illiquidity	ratio(21d), ratio(63d), ratio(252d), rank_liq, rank_iliq, thresh_high, thresh_low, trend4Q, zscore, vs-sector
D01	Aceleración momentum	ret63-ret126, ret21-ret63, ret126-ret252, ret63/ret126, señal_pos, zscore, rank, 2da-deriv, negativa, norm
D02	Score multi-timeframe	0.25×ret21+0.25×ret63+0.25×ret126+0.25×ret252, + 8 combos ponderaciones, top25%
D03	Consistencia retornos	%días pos 21d, 63d, 126d, 252d, >50%, >60%, >70%, tendencia-mejora, zscore, rank
D04	EMA vs SMA	EMA21/SMA21, EMA50/SMA50, EMA vs SMA, >1, <1, distancia, ratio-cruzado, señal,

z-score, rank

D05	Canal de precio	%channel(20d), %channel(63d), %channel(126d), >80%, >90%, <20%, <10%, breakout_up, breakout_dn, norm
-----	-----------------	---

E01	Fuerza relativa vs índice	ret - ret_index (21d, 63d, 126d, 252d), rank, >0%, >5%, >10%, zscore, rolling_rank, tendencia
-----	---------------------------	---

E02	Momentum sectorial	avg_ret_peers (21d, 63d, 126d), vs_sector, sector_rank, top2, bottom2, spread, zscore, rank, tendencia
-----	--------------------	--

E03	Tendencia de volumen	slope(21d), slope(63d), slope(126d), >0%, >+20%, >+50%, ratio(5/15d), zscore, señal-pos, rank
-----	----------------------	---

E04	Correlación con IPC	corr_63d, corr_126d, corr_252d, <0.3, >0.7, beta_63d, beta_126d, beta_252d, tracking-error, corr-inversa
-----	---------------------	--

E05	Tamaño relativo	rank(top5), rank(top3), decil, tercil, log_price30d, cambio_rank_YoY, stable_large, concentración, peso_eq, peso_prop
-----	-----------------	---

F01	Tasa real (Banxico-inf)	tasa-CPI_proxy, >5%, >6%, >7%, >8%, <4%, cambio_rate_real, zscore, rank, tendencia
-----	-------------------------	--

F02	Spread yield-CETES	div_yield - CETES_28d, >0%, >1%, >2%, >3%, >4%, zscore, rank, tendencia-spread, negativo
-----	--------------------	--

F03	Fase ciclo de tasas	subiendo(1/0), bajando(1/0), pausa(1/0), acelerando, desacelerando, cum_change_6M, velocidad, zscore, señal, dirección
-----	---------------------	--

F04	Tendencia USD/MXN	ret_usdmxn (21d, 63d, 126d, 252d), >0% peso-débil, <0% peso-fuerte, >+5%, <-5%, vol, corr-FIBRA, zscore
-----	-------------------	---

F05	Momentum IPC	ret_ipc (21d, 63d, 126d, 252d), >0%, >5%, >10%, <0%, zscore, rank_hist, tendencia, IPC>MA200(1/0), breadth_signal
-----	--------------	---

2. Estructura de archivos e implementación

```
src/
  features/
    registry.py      # FEATURE_REGISTRY: dict[int, FeatureDef] (400 features)
    builder.py       # build_feature_matrix(gene_ids, ...) -> DataFrame
  ml/
    models.py        # MODEL_REGISTRY: instanciar modelo por tipo+params
    cross_val.py      # walk_forward_cv(...) -> List[float]
    runner.py         # run_all_models(feature_matrix, labels) -> dict
  genetic/
    chromosome.py     # Gene, Individual dataclasses
    operators.py      # crossover(), mutate(), tournament_select()
    fitness.py        # evaluate_individual(ind, rebalance_data) -> float
    ga.py             # run_ga(config) -> GAResult
  app/pages/
    3_algoritmo.py    # Sección 2A (ML Puro) + Sección 2B (GA)
  results/
    ml_results.pkl    # Cache resultados 2A
    ga_results.pkl    # Cache resultado 2B
```

Orden de implementación

1. src/features/registry.py — 400 features con funciones compute
2. src/features/builder.py — build_feature_matrix()
3. src/ml/models.py + cross_val.py + runner.py — 9 modelos (2A)
4. src/genetic/chromosome.py — Gene, Individual
5. src/genetic/operators.py — crossover, mutate, select
6. src/genetic/fitness.py — evaluate_individual() sin ML (2B)
7. src/genetic/ga.py — run_ga()
8. app/pages/3_algoritmo.py — Sección 2A + Sección 2B
9. Actualizar requirements.txt (xgboost, catboost, lightgbm)
10. Smoke test: 3 gen × 5 individuos + 1 modelo ML → sin errores

Verificación

```
conda activate fibras-mx

# 2A: ML Puro
python -c "
from src.ml.runner import run_all_models
results = run_all_models()
print('Mejor modelo:', max(results, key=lambda k: results[k]['sharpe_test']))
"

# 2B: GA (smoke test rápido)
python -c "
```

```
from src.genetic.ga import run_ga
result = run_ga(n_generations=3, population_size=5)
print('Mejor Sharpe train:', result.best_individual.fitness)
print('Genes:', result.best_individual.genes)
print('Top-k:', result.best_individual.top_k)
"

# Streamlit
streamlit run app/main.py
# -> Pestaña 'Algoritmo': Sección 2A (ML Puro) | Sección 2B (GA)
```